

Маршрутизация в React

Введение в маршрутизацию

Маршрутизация — это механизм, который позволяет перемещаться между различными частями приложения без перезагрузки страницы. В одностраничных приложениях (SPA) маршрутизация особенно важна, так как она позволяет создавать впечатление многостраничного приложения, хотя на самом деле загружается только одна HTML-страница.

React Router

React Router — это стандартная библиотека маршрутизации для React. Она позволяет создавать навигацию в приложении React, синхронизируя пользовательский интерфейс с URL-адресом.

Установка React Router

```
npm install react-router-dom
```

Основные компоненты

BrowserRouter

BrowserRouter использует HTML5 History API для синхронизации UI с URL-адресом.

```
import { BrowserRouter } from 'react-router-dom';

function App() {
  return (
    <BrowserRouter>
      {/* Здесь размещается содержимое приложения */}
    </BrowserRouter>
  );
}
```

Routes и Route

Routes и **Route** используются для определения маршрутов в приложении.

```
import { BrowserRouter, Routes, Route } from 'react-router-dom';

function App() {
  return (
    <BrowserRouter>
```

```

    <Routes>
      <Route path="/" element={<Home />} />
      <Route path="/about" element={<About />} />
      <Route path="/contact" element={<Contact />} />
    </Routes>
  </BrowserRouter>
);
}

```

Link

Link используется для навигации между страницами без перезагрузки.

```

import { Link } from 'react-router-dom';

function Navigation() {
  return (
    <nav>
      <ul>
        <li>
          <Link to="/">Главная</Link>
        </li>
        <li>
          <Link to="/about">О нас</Link>
        </li>
        <li>
          <Link to="/contact">Контакты</Link>
        </li>
      </ul>
    </nav>
  );
}

```

NavLink

NavLink — это специальная версия **Link**, которая добавляет стили к активному элементу.

```

import { NavLink } from 'react-router-dom';

function Navigation() {
  return (
    <nav>
      <ul>
        <li>
          <NavLink
            to="/"
            className={({ isActive }) => isActive ? "active" : ""}
          >

```

```

        Главная
      </NavLink>
    </li>
    <li>
      <NavLink
        to="/about"
        className={({ isActive }) => isActive ? "active" : ""}
      >
        О нас
      </NavLink>
    </li>
    <li>
      <NavLink
        to="/contact"
        className={({ isActive }) => isActive ? "active" : ""}
      >
        Контакты
      </NavLink>
    </li>
  </ul>
</nav>
);
}

```

Вложенные маршруты

React Router позволяет создавать вложенные маршруты для организации сложной навигации.

```

import { BrowserRouter, Routes, Route, Outlet } from 'react-router-dom';

function App() {
  return (
    <BrowserRouter>
      <Routes>
        <Route path="/" element={<Layout />}>
          <Route index element={<Home />} />
          <Route path="about" element={<About />} />
          <Route path="contact" element={<Contact />} />
        </Route>
      </Routes>
    </BrowserRouter>
  );
}

function Layout() {
  return (
    <div>
      <Navigation />
      <Outlet /> { /* Здесь будут отображаться вложенные маршруты */ }
      <Footer />
    </div>
  );
}

```

```
    </div>
  );
}
```

Параметры URL

React Router позволяет определять динамические сегменты URL с помощью параметров.

```
import { BrowserRouter, Routes, Route, useParams } from 'react-router-dom';

function App() {
  return (
    <BrowserRouter>
      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/users" element={<Users />} />
        <Route path="/users/:userId" element={<UserDetails />} />
      </Routes>
    </BrowserRouter>
  );
}

function UserDetails() {
  const { userId } = useParams();
  return <div>Детали пользователя с ID: {userId}</div>;
}
```

Программная навигация

Для программной навигации (например, после отправки формы) можно использовать хук `useNavigate`.

```
import { useNavigate } from 'react-router-dom';

function LoginForm() {
  const navigate = useNavigate();

  const handleSubmit = (event) => {
    event.preventDefault();
    // Логика авторизации
    navigate('/dashboard'); // Перенаправление на страницу dashboard
  };

  return (
    <form onSubmit={handleSubmit}>
      {/* Поля формы */}
      <button type="submit">Войти</button>
    </form>
  );
}
```

```
    );  
  }
```

Защищенные маршруты

Часто требуется ограничить доступ к определенным маршрутам только для авторизованных пользователей.

```
import { Navigate, useLocation } from 'react-router-dom';  
  
function RequireAuth({ children }) {  
  const location = useLocation();  
  const isAuthenticated = checkIfUserIsAuthenticated(); // Ваша функция  
  проверки аутентификации  
  
  if (!isAuthenticated) {  
    // Перенаправление на страницу входа с сохранением исходного URL  
    return <Navigate to="/login" state={{ from: location }} replace />;  
  }  
  
  return children;  
}  
  
function App() {  
  return (  
    <BrowserRouter>  
      <Routes>  
        <Route path="/" element={<Home />} />  
        <Route path="/login" element={<Login />} />  
        <Route  
          path="/dashboard"  
          element={  
            <RequireAuth>  
              <Dashboard />  
            </RequireAuth>  
          }  
        />  
      </Routes>  
    </BrowserRouter>  
  );  
}
```

Альтернативные решения для маршрутизации

Reach Router

Reach Router был объединен с React Router v6, но его API может быть полезно знать, так как некоторые проекты все еще могут его использовать.

@tanstack/router (бывший React Location)

Это современная альтернатива React Router с типизацией TypeScript, асинхронной загрузкой данных и другими продвинутыми функциями.

```
import { Router, Route, Link } from '@tanstack/react-router';

// Определение маршрутов
const rootRoute = new Route({
  component: Root,
});

const indexRoute = new Route({
  getParentRoute: () => rootRoute,
  path: '/',
  component: Home,
});

const aboutRoute = new Route({
  getParentRoute: () => rootRoute,
  path: '/about',
  component: About,
});

// Создание роутера
const router = new Router({
  routeTree: rootRoute.addChildren([indexRoute, aboutRoute]),
});

function App() {
  return <RouterProvider router={router} />;
}
```

Заключение

Маршрутизация — важная часть современных React-приложений, которая позволяет создавать многостраничные приложения без перезагрузки страницы. React Router является наиболее популярным решением для маршрутизации в React, но существуют и альтернативные библиотеки с различными функциями и подходами.

При выборе решения для маршрутизации важно учитывать требования проекта, такие как необходимость вложенных маршрутов, параметров URL, программной навигации и защищенных маршрутов.